



RELATÓRIO DE DESENVOLVIMENTO

Aplicação de Gestão de Projetos e Tarefas

Trabalho Final | Módulo 5 | Integração Full Stack

Equipe 01

Allef Oliveira Ramos	@allef-oliveira
Camila Félix dos Reis	@cawzkf
Fernanda de Oliveira da Costa	@nanda-costa
Juliana Ballin Lima	@JulianaBallin
Pedro Henrique Oliveria Dias	@pedroddias-oss

Capacitação em IA e Transformação Digital

Identificação do documento

Projeto	OrbitBoard
Documento	Relatório de Desenvolvimento da Equipe 01
Curso	Capacitação em IA e Transformação Digital
Módulo	Módulo 5 de Integração Full Stack
Repositório	https://github.com/JulianaBallin/orbit-board-equipe-01
Branch de desenvolvimento	develop
Versão	1.0
Data de referência	20 de julho de 2026
Situação	Fundação técnica concluída e evidências finais em preparação

Controle de revisão

Este documento foi preparado para registrar a arquitetura, a integração, a infraestrutura, os ajustes técnicos, os testes e as evidências do OrbitBoard. As áreas marcadas como pendentes devem ser atualizadas pelos integrantes responsáveis antes da apresentação final.

Resumo executivo

O OrbitBoard é uma aplicação full stack para gestão de projetos, tarefas e responsabilidades de uma equipe. O frontend foi construído com React e Vite. O backend usa ASP.NET Core 8 e expõe uma API REST documentada por Swagger. A execução integrada ocorre por Docker Compose, com Nginx servindo o frontend e health checks para os dois serviços.

A equipe adotou a aplicação base fornecida pelo docente e concentrou o desenvolvimento na revisão da arquitetura, containerização, configuração de CORS, atualização de dependências, documentação, validação da comunicação HTTP e organização do fluxo Git. Os dados permanecem em memória para manter o foco didático na integração entre camadas.

Resultado técnico atual

- Backend compilado sem erros e sem avisos.
- Frontend compilado com Vite 8.1.5 e auditoria sem vulnerabilidades conhecidas.
- Docker Compose validado com backend e frontend saudáveis.
- Swagger, frontend e endpoints de saúde acessíveis nas portas documentadas.
- Pipeline do GitHub Actions aprovada para backend, frontend e containers.
- Roteiro com doze evidências preparado para execução final da equipe.

Sumário

Identificação do documento	i
Resumo executivo	ii
1 Contexto e objetivos	1
1.1 Contexto acadêmico	1
1.2 Objetivo geral	1
1.3 Objetivos específicos	1
1.4 Escopo adotado	2
2 Visão funcional	3
2.1 Descrição da aplicação	3
2.2 Funcionalidades	3
2.3 Fluxos principais	4
2.3.1 Cadastro de projeto	4
2.3.2 Cadastro e movimentação de tarefa	4
2.3.3 Tratamento de conflito	4
3 Arquitetura da solução	5
3.1 Visão de contêineres	5
3.2 Responsabilidades por camada	6
3.3 Fluxo HTTP e JSON	6
3.4 CORS	6
3.5 Persistência em memória	7
4 Tecnologias e organização	8

4.1	Tecnologias utilizadas	8
4.2	Estrutura do repositório	8
4.3	Decisões técnicas	9
5	Backend e contrato da API	11
5.1	Organização interna	11
5.2	Endpoints principais	11
5.3	Exemplo de criação de projeto	12
5.4	Validações e erros	12
6	Frontend e experiência de uso	13
6.1	Páginas	13
6.2	Cliente HTTP	13
6.3	Estados da interface	13
6.4	Publicação com Nginx	13
7	Docker, ambiente e automação	14
7.1	Containers	14
7.2	Variáveis de ambiente	14
7.3	Execução com Docker Compose	14
7.4	Menu de automação	15
8	Integração contínua e fluxo Git	16
8.1	Pipeline	16
8.2	Estratégia de branches	16
8.3	Padrão de commits	17
9	Ajustes técnicos realizados	18
9.1	Justificativa dos ajustes	18
10	Testes e resultados	20
10.1	Ambiente da verificação inicial	20
10.2	Resultados técnicos iniciais	20
10.3	Matriz de testes manuais	21

11 Evidências visuais	22
12 Equipe e contribuições	27
12.1 Integrantes	27
12.2 Frentes sugeridas	27
13 Plano de apresentação	28
13.1 Roteiro de 10 minutos	28
13.2 Sequência de demonstração	28
13.3 Plano de contingência	29
14 Limitações, riscos e próximos passos	30
14.1 Limitações conhecidas	30
14.2 Riscos para a apresentação	30
14.3 Próximos passos obrigatórios	30
14.4 Melhorias opcionais	31
15 Conclusão	32
A Comandos de referência	34
A.1 Fluxo rápido pelo Makefile	34
A.2 Execução local sem Docker	34
A.3 Fluxo Git	34
B Catálogo de evidências	36
C Atualização deste relatório	37

1

Contexto e objetivos

1.1 Contexto acadêmico

O trabalho final do Módulo 5 utiliza uma aplicação full stack como estudo de caso. A avaliação considera a compreensão da arquitetura, a organização do repositório, a integração entre frontend e backend, os ajustes técnicos, a execução por Docker, os testes, as evidências e a apresentação da equipe.

1.2 Objetivo geral

Preparar, executar, analisar, documentar e apresentar uma aplicação full stack que demonstre comunicação HTTP e JSON entre um frontend React e uma API ASP.NET Core, com ambiente reproduzível por Docker Compose.

1.3 Objetivos específicos

- Compreender a responsabilidade de cada camada da solução.
- Validar o consumo real da API pelo frontend.
- Documentar endpoints, contratos JSON, portas e variáveis de ambiente.
- Configurar CORS para permitir somente origens conhecidas.
- Criar imagens Docker reproduzíveis para frontend e backend.
- Automatizar builds por integração contínua.
- Executar testes positivos e cenários de erro.
- Registrar evidências visuais e logs sem dados sensíveis.
- Organizar contribuições por branches, commits e MRs.

1.4 Escopo adotado

A equipe escolheu a aplicação fornecida pelo docente. O escopo preserva os dados em memória e não inclui autenticação, banco de dados persistente ou pipeline de implantação. Essas escolhas mantêm o foco na integração full stack e atendem ao escopo mínimo do módulo.

Matriz do escopo obrigatório

Área	Entrega	Situação
Backend	API, Swagger, CORS, JSON, validação e erros	Aprovado
Frontend	Consumo da API, formulários, filtros e estados	Aprovado
Docker	Dois Dockerfiles, Compose, portas e health checks	Aprovado
Git	Branches, commits convencionais, CI e MRs	Aprovado
Documentação	README, arquitetura, contrato e roteiro	Aprovado
Evidências	Doze capturas planejadas e relatório preparado	Pendente
Apresentação	Roteiro de 10 minutos preparado	Pendente

2

Visão funcional

2.1 Descrição da aplicação

O OrbitBoard centraliza o acompanhamento de projetos e tarefas. A interface apresenta métricas, projetos, quadro de tarefas e integrantes. A API mantém as regras de negócio, valida os contratos e devolve respostas em JSON. O sistema inclui dados iniciais para facilitar a demonstração sem dependências externas.

2.2 Funcionalidades

Funcionalidade	Comportamento
Dashboard	Exibe projetos, tarefas, itens concluídos, atrasos, atividades recentes e distribuição por status.
Projetos	Permite cadastrar, consultar, editar e excluir projetos que não possuam tarefas vinculadas.
Tarefas	Permite cadastrar, consultar, editar, excluir e alterar rapidamente o status de uma tarefa.
Filtros	Restringe tarefas por projeto, status, prioridade, responsável e texto.
Equipe	Lista os integrantes fictícios cadastrados nos dados iniciais da aplicação.
Validação	Verifica campos obrigatórios, limites de texto, datas, referências e nomes duplicados.
Erros	Responde com <code>ProblemDetails</code> e apresenta mensagens compreensíveis no frontend.
Swagger	Disponibiliza documentação interativa dos endpoints.

Funcionalidade	Comportamento
Health checks	Permite verificar a disponibilidade do backend e do frontend.

2.3 Fluxos principais

2.3.1 Cadastro de projeto

O usuário abre a tela de projetos, preenche nome, descrição, status, datas e responsável. O frontend serializa o formulário em JSON e envia uma requisição `POST`. A API valida os dados, aplica regras de unicidade e devolve `201 Created`. O frontend então recarrega a listagem e apresenta uma mensagem de sucesso.

2.3.2 Cadastro e movimentação de tarefa

A tarefa é criada para um projeto existente. O usuário informa título, descrição, prioridade, responsável, prazo e estimativa. Depois da criação, a tarefa aparece no quadro. A mudança de coluna envia uma requisição `PATCH` contendo apenas o novo status.

2.3.3 Tratamento de conflito

Ao tentar criar um projeto com nome já existente ou excluir um projeto que ainda possui tarefas, a API responde com código `409`. O middleware padroniza a resposta e o frontend apresenta a mensagem sem perder o estado da tela.

3

Arquitetura da solução

3.1 Visão de contêineres

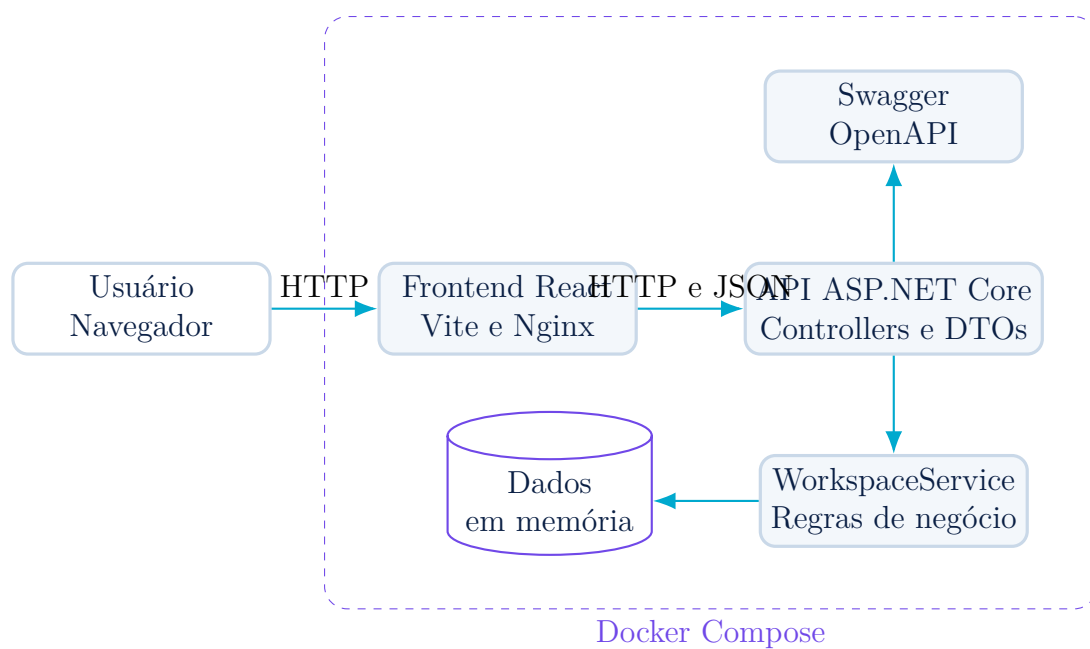


Figura 3.1: Arquitetura lógica e componentes executáveis do OrbitBoard.

3.2 Responsabilidades por camada

Camada	Responsabilidade	Tecnologia
Interface	Navegação, formulários, filtros, mensagens e apresentação do estado	React 18, React Router e Vite 8
Cliente HTTP	Montagem de URLs, serialização JSON e interpretação de erros	Fetch API
API	Endpoints, validação, CORS, Swagger e orquestração das regras	ASP.NET Core 8
Domínio	Projetos, tarefas, integrantes, enums e regras de consistência	C# e WorkspaceService
Dados	Armazenamento temporário e carga de exemplos	Coleções em memória
Infraestrutura	Build, execução, rede, portas e health checks	Docker, Compose e Nginx

3.3 Fluxo HTTP e JSON

1. O usuário executa uma ação na interface.
2. O cliente em `frontend/src/api/client.js` cria a requisição.
3. A URL base é obtida de `VITE_API_URL`.
4. O controller recebe o JSON e o ASP.NET Core valida o DTO.
5. O serviço aplica a regra de negócio e atualiza as coleções em memória.
6. A API devolve JSON, resposta sem conteúdo ou `ProblemDetails`.
7. O frontend atualiza a tela ou apresenta a mensagem correspondente.

3.4 CORS

O navegador abre o frontend em <http://localhost:5173> e acessa a API em <http://localhost:5200>. Como as origens são diferentes, a API usa a política `Frontend`. A lista de origens é lida da seção `Cors:AllowedOrigins`. No Compose, a variável `CORS_ALLOWED_ORIGIN` preenche `Cors__AllowedOrigins__0`.

3.5 Persistência em memória

O serviço usa listas protegidas por bloqueio de sincronização. Essa implementação é suficiente para o protótipo acadêmico, reduz a configuração inicial e permite demonstrar todas as operações HTTP. A limitação principal é a perda dos dados quando a API reinicia.

4

Tecnologias e organização

4.1 Tecnologias utilizadas

Tecnologia	Versão	Aplicação
.NET e ASP.NET Core	8	API REST, controllers, validação, middleware e health check
Swashbuckle	6.6.2	Geração do Swagger e OpenAPI
React	18.3	Interface e componentes
React Router	6.28	Navegação entre páginas
Vite	8.1.5	Servidor de desenvolvimento e build do frontend
Node.js	20.19 ou superior	Ambiente de build do frontend
Nginx	1.27	Publicação dos arquivos estáticos no container
Docker Compose	2.20 ou superior	Orquestração dos serviços
GitHub Actions	Actions oficiais	Integração contínua
LaTeX	pdfLaTeX	Geração deste relatório

4.2 Estrutura do repositório

```
orbit-board-equipe-01/  
|-- backend/  
|   |-- Dockerfile  
|   |-- OrbitBoard.Api.sln  
|   '-- OrbitBoard.Api/  
|       |-- Controllers/  
|       |-- DTOs/  
|       |-- Exceptions/
```

```
|      |-- Middleware/
|      |-- Models/
|      '-- Services/
|-- frontend/
|   |-- Dockerfile
|   |-- nginx.conf
|   '-- src/
|      |-- api/
|      |-- components/
|      |-- pages/
|      '-- utils/
|-- docs/
|   |-- assets/
|   |-- evidencias/
|   |   |-- capturas/
|   |   |-- relatorio-desenvolvimento-equipe01.tex
|   |   '-- relatorio-desenvolvimento-equipe01.pdf
|   |-- arquitetura.md
|   |-- contrato-api.md
|   |-- contribuicoes.md
|   |-- evidencias-testes.md
|   |-- fluxo-git.md
|   |-- registro-ajustes.md
|   '-- roteiro-apresentacao.md
|-- .env.example
|-- .github/workflows/ci.yml
|-- docker-compose.yml
|-- Makefile
'-- README.md
```

4.3 Decisões técnicas

Decisão	Justificativa
React com Vite	Oferece desenvolvimento rápido, componentes reutilizáveis e separação clara da API.
ASP.NET Core 8	Fornecer controllers, DTOs, validação, injeção de dependência, middleware e Swagger.
Dados em memória	Mantém o foco na integração e elimina a necessidade de configurar um banco de dados.
Nginx no frontend	Publica o build estático e mantém suporte às rotas da aplicação.
Dockerfiles em múltiplos estágios	Separam compilação e execução, reduzindo o conteúdo das imagens finais.

Decisão	Justificativa
CORS configurável	Permite alterar a origem autorizada sem modificar o código da API.
Health checks	Tornam o estado dos serviços observável e coordenam a inicialização do frontend.
Branches de integração	Mantêm <code>main</code> estável e concentram validações em <code>develop</code> .
Makefile	Padroniza comandos e reduz erros na execução por diferentes integrantes.

5

Backend e contrato da API

5.1 Organização interna

Os controllers expõem endpoints HTTP e delegam operações ao `IWorkspaceService`. Os DTOs definem contratos de entrada e saída. As entidades e enums ficam em `Models`. O middleware captura exceções e devolve `ProblemDetails`. O serviço singleton mantém os dados iniciais e aplica as regras de negócio.

5.2 Endpoints principais

Método	Endpoint	Finalidade
GET	/health	Verificar a saúde da API
GET	/api/dashboard	Consultar métricas e tarefas recentes
GET	/api/projects	Listar projetos
GET	/api/projects/{id}	Consultar um projeto
POST	/api/projects	Criar um projeto
PUT	/api/projects/{id}	Atualizar um projeto
DELETE	/api/projects/{id}	Excluir projeto sem tarefas
GET	/api/tasks	Listar e filtrar tarefas
GET	/api/tasks/{id}	Consultar uma tarefa
POST	/api/tasks	Criar uma tarefa
PUT	/api/tasks/{id}	Atualizar uma tarefa
PATCH	/api/tasks/{id}/status	Alterar somente o status
DELETE	/api/tasks/{id}	Excluir uma tarefa
GET	/api/team-members	Listar integrantes da aplicação

5.3 Exemplo de criação de projeto

```
POST /api/projects
Content-Type: application/json

{
  "name": "Integracao da equipe",
  "description": "Projeto para validar o fluxo completo.",
  "status": "Active",
  "startDate": "2026-07-20",
  "dueDate": "2026-08-20",
  "ownerId": "UUID_DO_INTEGRANTE"
}
```

5.4 Validações e erros

Código	Situação
400	Campo obrigatório ausente, tamanho inválido, data inconsistente ou referência inexistente
404	Projeto ou tarefa não encontrado
409	Projeto duplicado ou exclusão bloqueada pela existência de tarefas
500	Erro interno não tratado, registrado nos logs

```
{
  "title": "Conflito de regra",
  "status": 409,
  "detail": "Ja existe um projeto com esse nome.",
  "instance": "/api/projects",
  "traceId": "IDENTIFICADOR_DA_REQUISICAO"
}
```

6

Frontend e experiência de uso

6.1 Páginas

Página	Responsabilidade
Dashboard	Apresentar métricas, distribuição por status e tarefas recentes
Projetos	Listar, criar, editar e excluir projetos
Tarefas	Exibir o quadro, aplicar filtros e alterar o status
Equipe	Mostrar integrantes disponíveis nos dados da aplicação

6.2 Cliente HTTP

O arquivo `frontend/src/api/client.js` concentra a URL base, a serialização JSON, os métodos HTTP e a interpretação de respostas. Essa centralização reduz duplicação e garante que os erros da API sejam convertidos em mensagens consistentes para as páginas.

6.3 Estados da interface

As páginas apresentam carregamento durante as requisições, estado vazio quando não há registros, confirmação após operações bem-sucedidas e mensagens de erro quando a API rejeita a ação ou fica indisponível.

6.4 Publicação com Nginx

O build de produção gera arquivos estáticos na pasta `dist`. O Nginx publica esses arquivos na porta interna 80. A diretiva `try_files` devolve `index.html` para rotas da aplicação, permitindo que o React Router controle a navegação.

7

Docker, ambiente e automação

7.1 Containers

Serviço	Imagem final	Características
Backend	ASP.NET Core 8	API na porta 8080, usuário sem privilégios e health check por <code>curl</code>
Frontend	Nginx 1.27 Alpine	Arquivos estáticos na porta 80 e health check por <code>wget</code>

7.2 Variáveis de ambiente

Variável	Padrão	Finalidade
BACKEND_PORT	5200	Porta pública da API
FRONTEND_PORT	5173	Porta pública do frontend
VITE_API_URL	http://localhost:5200	URL da API usada pelo navegador
CORS_ALLOWED_ORIGIN	http://localhost:5173	Origem aceita pela política de CORS

7.3 Execução com Docker Compose

```
cp .env.example .env
docker compose up --build -d
docker compose ps
docker compose logs --follow
docker compose down
```

O frontend depende do health check do backend. Essa condição evita que a interface seja iniciada antes que a API esteja disponível.

7.4 Menu de automação

O `Makefile` oferece uma interface única para os comandos mais importantes. O menu é aberto com:

```
make menu
```

Comando	Resultado
<code>make setup</code>	Prepara <code>.env</code> e instala dependências
<code>make build</code>	Compila backend e frontend
<code>make up</code>	Compila e inicia os containers
<code>make down</code>	Encerra os containers
<code>make status</code>	Exibe a saúde e as portas publicadas
<code>make logs</code>	Acompanha os logs dos serviços
<code>make health</code>	Consulta os health checks
<code>make validate</code>	Executa builds, auditoria e validação do Compose
<code>make report</code>	Compila este relatório e incorpora capturas disponíveis

8

Integração contínua e fluxo Git

8.1 Pipeline

A pipeline é executada em pushes e MRs destinados a **develop** e **main**. Ela possui três jobs:

1. **Build backend**: restaura pacotes e compila a solução em modo Release.
2. **Build frontend**: instala dependências com **npm ci** e executa o build.
3. **Build containers**: valida o Compose e compila as duas imagens.

O job de containers depende da aprovação dos builds das aplicações. Isso impede a criação das imagens quando alguma camada não compila.

8.2 Estratégia de branches

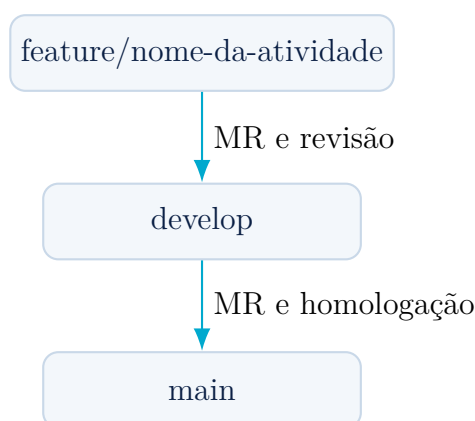


Figura 8.1: Fluxo de integração adotado pela equipe.

Não são feitos commits diretamente em **main** ou **develop**. Cada alteração nasce em uma branch de trabalho, recebe uma mensagem convencional em inglês e entra em **develop** por

MR. Depois da validação integrada, `develop` é promovida para `main` por outro MR.

8.3 Padrão de commits

```
feat(tooling): add interactive project menu
fix(frontend): update build dependencies
docs(project): add architecture and delivery guides
ci(build): validate application and container builds
```


9

Ajustes técnicos realizados

Item	Situação inicial	Ajuste
Backend em container	Apenas execução local	Dockerfile em múltiplos estágios com imagem final ASP.NET Core
Frontend em container	Apenas execução com Vite	Build em Node.js e publicação por Nginx
Integração	Sem Compose	Compose com portas, rede, dependência e health checks
CORS	Origem fixa no código	Origens carregadas da configuração e do ambiente
Ambiente	Variável apenas no frontend	Arquivo central <code>.env.example</code>
Dependências	Vite 5 com alertas	Vite 8.1.5 e plugin React compatível
Integração contínua	Ausente	Pipeline com três jobs e actions atuais
Documentação	READMEs da base	Guia completo e documentos especializados
Automação local	Comandos manuais dispersos	Makefile com menu e validação completa
Evidências	Sem estrutura de capturas	Catálogo, relatório e espaços automáticos para imagens

9.1 Justificativa dos ajustes

Os ajustes reduzem diferenças entre os ambientes dos integrantes, tornam os comandos reproduzíveis, registram decisões e facilitam a demonstração. A configuração externa evita alterações de código para trocar portas ou origens. A pipeline detecta falhas antes da

integração. O relatório concentra evidências e explicações em um documento único.

10

Testes e resultados

10.1 Ambiente da verificação inicial

Ferramenta	Versão utilizada
Sistema operacional	Ubuntu 24.04
Docker	28.5.1
Docker Compose	2.40.3
.NET SDK	8.0.415
Node.js	20.19.5
npm	10.8.2

10.2 Resultados técnicos iniciais

Verificação	Resultado	Situação
Build Release do backend	Sem erros e sem avisos	Aprovado
Build do frontend	Vite 8.1.5 concluído	Aprovado
Auditoria npm	Nenhuma vulnerabilidade	Aprovado
Validação do Compose	Configuração aceita	Aprovado
Containers	Dois serviços saudáveis	Aprovado
Frontend	Resposta HTTP 200	Aprovado
Swagger	Resposta HTTP 200	Aprovado
CORS	Origem do frontend autorizada	Aprovado

Verificação	Resultado	Situação
Dashboard	JSON com métricas iniciais	Aprovado
Criação de projeto	Operação concluída	Aprovado
Nome duplicado	Resposta HTTP 409	Aprovado
Exclusão temporária	Resposta HTTP 204	Aprovado

10.3 Matriz de testes manuais

ID	Cenário	Resultado esperado
E01	Iniciar a solução com Docker Compose	Dois serviços saudáveis
E02	Abrir o dashboard	Métricas e tarefas exibidas
E03	Abrir o Swagger	Endpoints documentados
E04	Consultar <code>/health</code>	JSON com status saudável
E05	Criar um projeto válido	Resposta 201 e item na interface
E06	Repetir o nome do projeto	Resposta 409 e mensagem clara
E07	Criar uma tarefa	Resposta 201 e item no quadro
E08	Aplicar filtros	Lista compatível com os critérios
E09	Alterar o status	Tarefa movida e resposta 200
E10	Excluir projeto com tarefa	Resposta 409 sem exclusão
E11	Interromper o backend	Mensagem de indisponibilidade
E12	Consultar logs	Inicialização sem erro não tratado

11

Evidências visuais

As imagens desta seção são carregadas da pasta `docs/evidencias/capturas/`. Enquanto um arquivo não existe, o relatório apresenta um espaço reservado com o nome exato esperado. Depois de adicionar as capturas, execute `make report` para atualizar o PDF.

Evidência aguardando captura

`E01-containers-saudaveis.png`

Adicione a imagem em `docs/evidencias/capturas/` e execute `make report`.

Figura 11.1: Containers do backend e frontend com health checks aprovados. Situação atual: Pendente de captura final.

Evidência aguardando captura

`E02-dashboard.png`

Adicione a imagem em `docs/evidencias/capturas/` e execute `make report`.

Figura 11.2: Dashboard carregado com métricas e tarefas recentes. Situação atual: Pendente de captura final.

Evidência aguardando captura

E03-swagger.png

Adicione a imagem em docs/evidencias/capturas/ e execute `make report`.

Figura 11.3: Swagger com os endpoints da API. Situação atual: Pendente de captura final.

Evidência aguardando captura

E04-health.png

Adicione a imagem em docs/evidencias/capturas/ e execute `make report`.

Figura 11.4: Resposta JSON do health check do backend. Situação atual: Pendente de captura final.

Evidência aguardando captura

E05-criar-projeto.png

Adicione a imagem em docs/evidencias/capturas/ e execute `make report`.

Figura 11.5: Criação de um projeto válido pela interface. Situação atual: Pendente de captura final.

Evidência aguardando captura

E06-projeto-duplicado.png

Adicione a imagem em docs/evidencias/capturas/ e execute `make report`.

Figura 11.6: Tratamento de conflito para nome duplicado. Situação atual: Pendente de captura final.

Evidência aguardando captura

E07-criar-tarefa.png

Adicione a imagem em docs/evidencias/capturas/ e execute `make report`.

Figura 11.7: Criação de uma tarefa vinculada a um projeto. Situação atual: Pendente de captura final.

Evidência aguardando captura

E08-filtrar-tarefas.png

Adicione a imagem em docs/evidencias/capturas/ e execute `make report`.

Figura 11.8: Aplicação de filtros no quadro de tarefas. Situação atual: Pendente de captura final.

Evidência aguardando captura

[E09-alterar-status.png](#)

Adicione a imagem em docs/evidencias/capturas/ e execute `make report`.

Figura 11.9: Alteração do status de uma tarefa. Situação atual: Pendente de captura final.

Evidência aguardando captura

[E10-bloqueio-exclusao.png](#)

Adicione a imagem em docs/evidencias/capturas/ e execute `make report`.

Figura 11.10: Bloqueio da exclusão de projeto com tarefas. Situação atual: Pendente de captura final.

Evidência aguardando captura

[E11-backend-indisponivel.png](#)

Adicione a imagem em docs/evidencias/capturas/ e execute `make report`.

Figura 11.11: Tratamento do frontend quando a API fica indisponível. Situação atual: Pendente de captura final.

Evidência aguardando captura

`E12-logs-containers.png`

Adicione a imagem em `docs/evidencias/capturas/` e execute `make report`.

Figura 11.12: Logs de inicialização dos containers. Situação atual: Pendente de captura final.

12

Equipe e contribuições

12.1 Integrantes

Integrante	GitHub	Situação do registro
Allef Oliveira Ramos	@allef-oliveira	Pendente
Camila Félix dos Reis	@cawzkf	Pendente
Fernanda de Oliveira da Costa	@nanda-costa	Pendente
Juliana Ballin Lima	@JulianaBallin	Em atualização
Pedro Henrique Oliveria Dias	@pedroddias-oss	Pendente

12.2 Frentes sugeridas

Integrante	Frente inicial
Allef	Testes da API e cenários de erro
Camila	Validação do frontend e melhoria de usabilidade
Fernanda	Docker Compose, logs e evidências de infraestrutura
Juliana	Organização do repositório, automação e documentação
Pedro	Arquitetura e roteiro da apresentação

Regra de autoria

A tabela final de contribuições deve refletir commits e MRs reais. Nenhum integrante deve registrar trabalho em nome de outra pessoa. Cada contribuição precisa ser associada ao usuário correto no GitHub.

13

Plano de apresentação

13.1 Roteiro de 10 minutos

Tempo	Tema	Conteúdo
0:00 a 1:00	Abertura	Nome, objetivo didático e integrantes
1:00 a 2:30	Arquitetura	Frontend, API, dados e Docker
2:30 a 4:30	Demonstração	Dashboard, projeto, tarefa, filtro e status
4:30 a 5:45	API	Swagger, JSON, health check e erros
5:45 a 7:00	Infraestrutura	Dockerfiles, Compose, portas, variáveis e logs
7:00 a 8:30	Testes	Cenários positivos, conflito e indisponibilidade
8:30 a 9:30	Ajustes	Melhorias realizadas e justificativas
9:30 a 10:00	Encerramento	Contribuições, limitações e próximos passos

13.2 Sequência de demonstração

1. Exibir `docker compose ps` com os serviços saudáveis.
2. Abrir o dashboard e relacionar as métricas à resposta da API.
3. Criar um projeto e mostrar a requisição no navegador.
4. Criar uma tarefa, aplicar filtros e alterar seu status.
5. Abrir o Swagger e executar uma consulta.
6. Demonstrar um conflito ao repetir o nome do projeto.
7. Mostrar os logs dos containers.
8. Encerrar com as decisões técnicas e contribuições.

13.3 Plano de contingência

As capturas deste relatório servem como apoio caso o ambiente não possa ser iniciado durante a apresentação. A equipe deve manter as imagens Docker prontas, verificar as portas e abrir previamente o frontend, o Swagger e um terminal com os comandos principais.

14

Limitações, riscos e próximos passos

14.1 Limitações conhecidas

- Os dados são apagados quando a API reinicia.
- Não existe autenticação ou autorização.
- A URL da API é incorporada ao frontend durante o build.
- O estado em memória não permite escalar o backend horizontalmente.
- A aplicação foi projetada para uso didático e execução local.

14.2 Riscos para a apresentação

Risco	Mitigação
Porta ocupada	Encerrar processos anteriores ou ajustar o arquivo <code>.env</code>
Imagem não compilada	Executar <code>make up</code> antes da apresentação
Backend indisponível	Conferir <code>make status</code> , <code>make logs</code> e o health check
Falha de rede	Manter capturas e PDF disponíveis localmente
Dados inconsistentes	Reiniciar o backend para restaurar os dados iniciais
Tempo excedido	Ensaiai o roteiro e limitar a demonstração aos fluxos principais

14.3 Próximos passos obrigatórios

1. Executar os doze testes manuais em ambiente da equipe.
2. Adicionar as capturas com os nomes definidos.

3. Recompilar este relatório com `make report`.
4. Registrar as contribuições reais dos cinco integrantes.
5. Definir os responsáveis por cada parte da apresentação.
6. Ensaiar e confirmar duração entre 8 e 12 minutos.
7. Integrar as alterações em `develop` por MR.
8. Promover a versão validada de `develop` para `main` por MR.

14.4 Melhorias opcionais

- Adicionar testes automatizados para regras da API.
- Criar testes de componentes e fluxos do frontend.
- Persistir os dados em um banco executado por container.
- Adicionar autenticação e autorização por perfis.
- Publicar uma imagem versionada em um registro de containers.
- Configurar monitoramento básico e métricas de aplicação.

Conclusão

O OrbitBoard demonstra os elementos centrais de uma aplicação full stack: interface web, API REST, comunicação HTTP e JSON, validação, tratamento de erros, CORS, documentação OpenAPI e execução containerizada. A infraestrutura criada permite reproduzir o ambiente com poucos comandos e a pipeline reduz o risco de integrar alterações que não compilam.

A documentação registra a arquitetura, as decisões, os contratos, os ajustes e o processo de trabalho. O Makefile facilita a execução por todos os integrantes. A estrutura de evidências e este relatório tornam a preparação final objetiva, pois cada imagem pendente possui nome, cenário e resultado esperado.

O estágio técnico está avançado. A conclusão depende principalmente da participação real dos integrantes, da produção das capturas, da atualização das contribuições e do ensaio da apresentação.

Checklist final

Item	Situação
Repositório organizado	Aprovado
README completo	Aprovado
Backend e Swagger	Aprovado
Frontend integrado	Aprovado
Dockerfiles e Compose	Aprovado
Pipeline de integração contínua	Aprovado
Makefile com menu	Aprovado
Relatório técnico	Aprovado
Capturas finais	Pendente
Contribuições dos integrantes	Pendente
Ensaio da apresentação	Pendente
MR final para main	Pendente

A

Comandos de referência

A.1 Fluxo rápido pelo Makefile

```
make menu
make setup
make validate
make up
make health
make logs
make down
make report
```

A.2 Execução local sem Docker

```
# Terminal 1
dotnet run --project backend/OrbitBoard.Api

# Terminal 2
npm ci --prefix frontend
npm run dev --prefix frontend
```

A.3 Fluxo Git

```
git switch develop
git pull origin develop
git switch -c feature/nome-da-atividade
git add .
git commit -m "feat(scope): describe the change"
git push -u origin feature/nome-da-atividade
```


B

Catálogo de evidências

Arquivo	Conteúdo
E01-containers-saudaveis.png	Containers saudáveis
E02-dashboard.png	Dashboard com métricas
E03-swagger.png	Endpoints no Swagger
E04-health.png	Resposta do health check
E05-criar-projeto.png	Projeto criado
E06-projeto-duplicado.png	Conflito de nome
E07-criar-tarefa.png	Tarefa criada
E08-filtrar-tarefas.png	Filtros aplicados
E09-alterar-status.png	Mudança de status
E10-bloqueio-exclusao.png	Exclusão bloqueada
E11-backend-indisponivel.png	Erro de conexão tratado
E12-logs-containers.png	Logs dos serviços



Atualização deste relatório

Para substituir os espaços reservados por capturas reais:

1. Salve os arquivos em `docs/evidencias/capturas/`.
2. Mantenha os nomes definidos no catálogo.
3. Revise as imagens para remover dados sensíveis.
4. Execute `make report` na raiz do projeto.
5. Abra o PDF e confira enquadramento, legibilidade e paginação.
6. Atualize o status das evidências no arquivo LaTeX e em `docs/evidencias-testes.md`.

Resultado esperado

Quando a imagem existe, a macro de evidência a incorpora automaticamente. Quando não existe, o relatório mantém um quadro informativo com o nome esperado, permitindo que o PDF seja compilado em qualquer estágio do trabalho.